Reliable and Secure ReRAM Crossbar Accelerators for Deep Neural Networks

Noise-, Fault- and Attack-Aware Hybrid SLC/MLC In-Memory Computing

Abstract. Resistive RAM (ReRAM) crossbars perform matrix–vector multiplication in place, removing the weight movement that dominates the energy of deep neural network (DNN) inference. Their practical use is limited by device non-idealities (conductance variation, stuck-at faults, limited multi-level precision, ADC quantisation and IR drop) and by the exposure of in-memory weights to fault-injection attacks. Building on HyFlexPIM, which places important weights in reliable single-level cells (SLC) and the remainder in dense multi-level cells (MLC), we will (i) develop and verify a bit-accurate crossbar simulator, (ii) reproduce the key published baselines, (iii) characterise the impact of each non-ideality on CNNs and Transformers, (iv) recover accuracy through noise-aware training, noise-aware mapping and fault-aware mapping, and (v) propose a *unified rank-to-slot mapping* that jointly accounts for cell tier, stuck-at faults and IR drop at zero runtime hardware cost. We further evaluate a multi-level-cell bit-flip attack and an analog checksum defence, quantify all overheads with NeuroSim, and emulate the accelerator on an AMD Kria KV260 FPGA.

1 Papers Included in the Project

The project is anchored on one primary paper, extends four foundational works named in our original scope, and draws on a small set of recent papers for baselines and techniques (Table 1).

Table 1: Papers included in the project and their role.

Role	Paper	What we use
Primary	HyFlexPIM, Song *et al.*, ISCA 2025	Hybrid SLC/MLC analog PIM; SVD-based gradient redistribution; importance-based tier mapping. Reproduced and extended.
Foundational architecture	ISAAC, Shafiee *et al.*, ISCA 2016	Reference crossbar organisation (tiles, bit-slicing, ADC sharing); PPA baseline.
	PRIME, Chi *et al.*, ISCA 2016	ReRAM main-memory PIM organisation; signed-weight mapping.
Reliability baseline	Xia *et al.*, DAC 2017	Fault-tolerant training and remapping for stuck-at faults; baseline for our mapping.
Security baseline	Rakin *et al.*, ICCV 2019	Bit-Flip Attack (progressive bit search); reproduced on crossbar-stored weights.
Supporting	Rasch *et al.*, Nat. Commun. 2023	Hardware-aware (noise-aware) training baseline.
	Chen *et al.*, DATE 2017	Optimal (bipartite) fault-aware weight mapping baseline.
	FREEMAP, ISLPED 2026	Zero-overhead folding of permutations into adjacent layers.
	SWIM, DAC 2022	Hessian-based weight-importance metric.
	Liu *et al.*, ITC 2018; RADAR, DATE 2021	Checksum-based error and attack detection.
	Luo *et al.*, TCAD 2020	FPGA emulation of ReRAM crossbars with noise.

2 Project Overview

Problem. A ReRAM crossbar stores weights as conductances G_{ij}; applying input voltages V_i to the rows yields column currents $I_j = \sum_i V_i G_{ij}$, i.e. an analog dot product. Real devices deviate from their programmed values, some cells are permanently stuck, multi-level cells have narrow noise margins, and wire resistance attenuates currents. Because weights reside in memory, an adversary able to perturb a few cells can also collapse model accuracy.

Gap in the primary paper. HyFlexPIM models device noise only as i.i.d. multiplicative Gaussian weight noise; it does not consider stuck-at faults, IR drop or drift, applies no noise-aware training, ignores *which physical cells* hold a given weight, provides no security analysis, and is evaluated on Transformers only.

Objectives.

O1 Build a bit-accurate, verified ReRAM crossbar simulator and reproduce published baselines.

O2 Characterise the accuracy impact of each non-ideality, individually and combined, on CNNs and Transformers.

O3 Recover accuracy via noise-aware training, noise-aware mapping and fault-aware mapping.

O4 Secure crossbar-stored weights against bit-flip attacks at low overhead.

O5 Quantify the energy, latency and area cost of every technique and demonstrate an FPGA emulator.

Planned contributions.

C1 **Unified rank-to-slot mapping.** A single optimal assignment that places each SVD component in a physical crossbar slot, jointly considering cell tier (SLC/MLC), stuck-at faults and IR-drop position, with no runtime permutation hardware.

C2 **Tier-aware noise-aware training.** Fine-tuning with SLC-level noise on protected components and MLC-level noise elsewhere, to reduce the required SLC fraction.

C3 **Multi-level-cell bit-flip threat model.** Attack cost measured in conductance-level shifts rather than bit flips, and the effect of level encoding (binary vs. Gray).

C4 **Analog checksum defence.** A per-crossbar checksum column evaluated against benign noise (false alarms) and an adaptive, checksum-aware attacker.

Scope. Two model tracks share one simulator: a *CNN track* (ResNet-20, ResNet-18 and VGG-16 on CIFAR-10) for rapid iteration and FPGA deployment, and a *Transformer track* (BERT-Base on GLUE SST-2 and MRPC) for direct comparison with HyFlexPIM.

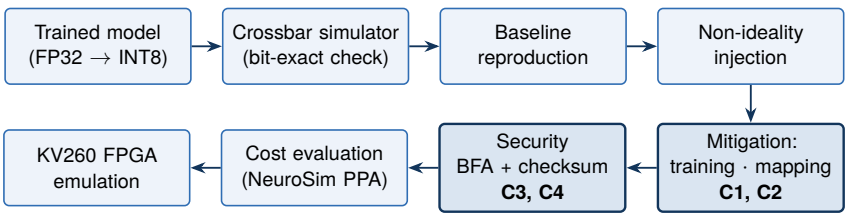


Figure 1: Project workflow. Highlighted stages contain our contributions.

3 Methodology

3.1 Step 1 – Baseline Models and Quantisation

We train FP32 baselines (ResNet-20, ResNet-18, VGG-16 on CIFAR-10; BERT-Base fine-tuned on SST-2 and MRPC) in PyTorch on Google Colab, then apply INT8 quantisation-aware training with symmetric per-channel weight scales. The INT8 models are the accuracy reference for all later steps.

3.2 Step 2 – Bit-Accurate Crossbar Simulator

We implement a PyTorch library in which every linear and convolutional layer (the latter via *im2col*) is replaced by a crossbar layer modelling the analog computation explicitly:

- **Signed weights** are stored as differential pairs, $w_{ij} \propto G^{+}_{ij} - G^{-}_{ij}$.
- **Bit-slicing:** an 8-bit weight is split into slices $w = \sum_s 2^{sb}\, w^{(s)}$, using b=1 bit per cell (SLC, 8 cells) or b=2 (MLC, 4 cells), tiled into 128×128 arrays.
- **Bit-serial inputs, ADC and shift-and-add:**

$$y_j = \sum_t \sum_s 2^{\,t+sb}\ \mathrm{ADC}\Big(\textstyle\sum_i x_i^{(t)}\, w_{ij}^{(s)}\Big), \tag{1}$$

  where $x_i^{(t)}$ is the t-th input bit.

Verification. With all non-idealities disabled and a lossless ADC of $b_{\mathrm{ADC}} = \lceil \log_2(R(2^b-1)+1) \rceil$ bits (R rows), the simulator must reproduce the INT8 model *bit-exactly* on every layer and in end-to-end accuracy. This test is automated and re-run after every change; it also makes the simulator the golden model for the FPGA (Step 8).

3.3 Step 3 – Reproduction of Baselines

- **HyFlexPIM:** using the authors' open-source code, reproduce accuracy versus SLC fraction on BERT-Base and the comparison of gradient-, rank- and magnitude-based selection.
- **HyFlexPIM on CNNs (new):** apply the same SVD and fine-tuning to convolutional layers and test whether importance concentrates in a few components, as it does for Transformers.
- **ISAAC/PRIME-style PPA:** configure DNN+NeuroSim to report energy, latency, area, TOPS/W and the ADC energy share, against a digital baseline.
- **Bit-Flip Attack:** reproduce the attack of Rakin *et al.* with the public implementation and record the number of flips required to reach random-guess accuracy.

3.4 Step 4 – Non-Ideality Modelling and Characterisation

Each effect is an independent, seeded module (Table 2). Parameters are calibrated to measured ReRAM data through the noise models of IBM AIHWKIT, and our simulator is validated against AIHWKIT and Sandia CrossSim (the latter for IR drop). Every experiment is repeated over at least 20 simulated chips, reporting mean, 5th-percentile and worst-case accuracy, together with a per-layer sensitivity analysis.

Table 2: Modelled non-idealities.

Effect	Model	Sweep
Programming variation	$\tilde{G} = G + \mathcal{N}(0, \sigma^2(G))$, fixed per chip	$\sigma / G_{\max} \in [0, 10]\%$
Read noise	Fresh Gaussian perturbation at every inference	0.5–2%
Stuck-at faults	Cells forced to $G_{\max}$ or $G_{\min}$; uniform and row/column-clustered	0.1–20% of cells
Limited levels	1–4 bits/cell; reduced level margin for MLC	SLC vs. MLC
ADC resolution	Clamp and round of column partial sums	3–8 bits
IR drop	Position-dependent attenuation along word/bit lines	$r_{\text{wire}}\times$ array size (64–256)
Drift (optional)	Time-dependent conductance relaxation	1 s to 1 week

3.5 Step 5 – Accuracy Recovery

5a. Noise-aware training (NAT). We train $\min_\theta \mathbb{E}_\eta \left[\mathcal{L}\left(f(x; \theta \odot (1+\eta))\right)\right]$, resampling noise every mini-batch with straight-through quantisation. Methods are evaluated incrementally: (i) Gaussian noise injection; (ii) the hardware-aware training recipe of Rasch *et al.* via AIHWKIT (weight clipping, learnable input range, ADC in the loop), our principal training baseline; (iii) stuck-at fault injection during training; (iv) worst-case-oriented objectives such as right-censored noise.

5b. SVD and noise-aware mapping. Following HyFlexPIM, each weight matrix is factorised and truncated, $W \approx U_k \Sigma_k V_k^\top$ with $k = \lfloor mn/(m+n) \rfloor$, which preserves the parameter count. After 1–3 epochs of fine-tuning, the importance of component r is measured as $I_r = \left|\partial \mathcal{L} / \partial \sigma_r\right|$; the top-$p$% components are placed in SLC and the rest in MLC. We compare gradient, magnitude, Hessian (SWIM) and Fisher importance at the weight, component and layer levels, and select per-layer SLC fractions under a NeuroSim area/energy budget.

Contribution C2 – Tier-aware noise-aware training

During HyFlexPIM's fine-tuning, components currently assigned to SLC receive SLC-level noise and the remaining components MLC-level noise, $\eta_r \sim \mathcal{N}\left(0, \sigma^2_{\tau(r)}\right)$, with the assignment τ refreshed periodically. The network thus learns to concentrate information where it is protected. *Hypothesis:* equal accuracy at a lower SLC fraction.

5c. Fault-aware mapping. Given a chip's stuck-at fault map, logical rows are assigned to physical rows by solving the linear assignment problem

$$\pi^\star = \arg\min_\pi \sum_i C_{i,\pi(i)}, \qquad C_{ij} = \sum_k I_{ik} \left(w_{ik} - \hat{w}_{ik}^{(j)}\right)^2, \tag{2}$$

where $\hat{w}_{ik}^{(j)}$ is the value realised when row i is stored on physical row j. The problem is solved optimally with the Hungarian algorithm; row and column permutations are alternated. Output-column permutations are folded into the next layer's input order (as in FREEMAP), so no permutation hardware is required. Spare rows, allocated by importance, handle clustered (dead-row) faults. Baselines: no mitigation, Xia *et al.*, Chen *et al.* and fault-injection training.

Contribution C1 – Unified rank-to-slot mapping

In an SVD-factorised layer $W = U(\Sigma V^\top)$ the k components form an internal dimension that can be permuted (columns of $\Sigma V^\top$ and rows of U together) without changing the layer output. Each physical slot s is characterised by its tier $\tau(s) \in \{\text{SLC}, \text{MLC}\}$, its fault pattern F_s and its IR-drop attenuation α_s. We solve

$$\min_{\pi} \sum_{r} I_r \, \mathbb{E}\left[\|\Delta_r(\pi(r))\|^2 \,\middle|\, \tau, F, \alpha\right] \quad \text{s.t. SLC capacity budget,}$$

unifying tier allocation (HyFlexPIM), fault-aware remapping and IR-drop-aware placement in a single optimal assignment with zero runtime hardware overhead. An ablation isolates each cost term.

5d. Complementary techniques (time permitting). Drift compensation, sign-flip weight storage matched to fault patterns, activation clipping, and reduced ADC precision for MLC arrays.

3.6 Step 6 – Hardware Cost Evaluation

All configurations are evaluated in DNN+NeuroSim V1.4 (CNNs) and NeuroSim V1.5 (Transformers) for energy per inference, latency, area, TOPS/W and TOPS/mm^2, with an energy breakdown (ADC, array, peripherals, buffers). The overhead of each technique (additional SLC cells, spare rows, checksum columns) is reported. The central result is the *accuracy–efficiency Pareto front* obtained by varying the SLC fraction.

3.7 Step 7 – Security

Threat model. The adversary shifts cell conductance levels through physical disturbance (thermal crosstalk, write or read disturbance), restricted to cells physically reachable from the attacker's access pattern.

Contributions C3 and C4 – MLC-aware attack and analog checksum defence

C3. The Bit-Flip Attack is reformulated with cost measured in conductance-level shifts: in a 2-bit cell one shift may flip two bits. We test whether MLC lowers attack cost and whether Gray-coded levels remove this advantage.

C4. Each crossbar holds a checksum column $G_{i,\text{chk}} = \sum_j G_{ij}$; tampering is flagged when $\left|\sum_j I_j - I_{\text{chk}}\right| > \theta$, with θ set from the benign-noise distribution (cost $\approx 1/128$ columns). Unlike prior checksum schemes for random faults (X-ABFT) or digital weights (RADAR), we evaluate against an adaptive attacker that preserves the checksum, countered by a secret-weighted checksum.

Evaluation: flips or level shifts required to reach random-guess accuracy, with and without each defence (noise-aware training, SLC protection, Gray coding, checksum); detection ROC and AUC; detection latency; recovery by row reprogramming.

3.8 Step 8 – FPGA Emulation on AMD Kria KV260

The emulator implements the arithmetic of the simulator exactly, so that the simulator serves as its golden model.

- **Processing system (ARM, PYNQ):** loads conductance codes and stuck-at masks, configures ADC precision and noise level through AXI-Lite, and streams activations via DMA.
- **Programmable logic (Vitis HLS):** several 128×128 virtual-crossbar tiles, each with on-chip weight storage, parallel MAC lanes, hardware Gaussian noise generators (Tausworthe-based, following Luo *et al.*), ADC rounding, shift-and-add and an optional checksum column.

- **Mapping:** fault-aware and rank permutations are folded into the stored weights and require no logic, directly demonstrating contribution C1's zero-overhead property.
- **Workload:** ResNet-20 (0.27 M weights) resides entirely on-chip.
- **Validation and metrics:** bit-exact agreement with noise disabled, statistical agreement with noise enabled, speed-up over the software simulator for Monte Carlo sweeps, resource utilisation, achieved clock frequency and board power.

4 Tools and Frameworks

Table 3: Software and hardware tools.

Purpose	Tool	Role in the project
Core simulation	Custom PyTorch library	Crossbar model, non-idealities, SLC/MLC tiers, mappings; FPGA golden model
Analog reference	IBM AIHWKIT 1.1	Calibrated ReRAM noise models, simulator validation, NAT baseline
IR-drop validation	Sandia CrossSim	Circuit-level parasitic solver
Primary baseline	HyFlexPIM code	SVD and gradient-redistribution pipeline
Energy, latency, area	DNN+NeuroSim V1.4 / V1.5	PPA of all configurations
Optimisation	SciPy, PuLP / OR-Tools	Hungarian assignment and ILP
Security	Public BFA implementation	Attack reproduction and extension
FPGA	Vitis HLS, Vivado, Kria-PYNQ	KV260 emulator (Linux x86 host required)
Compute	Google Colab (GPU), laptops	Training and Monte Carlo sweeps